RELATÓRIO TÉCNICO: WEB PROXY COM CONTROLE DE CONTEÚDO

UNIVERSIDADE FEDERAL DO RIO GRANDE – FURG
CENTRO DE CIÊNCIAS COMPUTACIONAIS (C3)
DISCIPLINA: SISTEMAS PARA INTERNET 2 (2026/1)

Informação	Detalhe
Desenvolvimento	Matheus Siqueira Pinto (Engenharia de Computação)
Tecnologia	Python 3.12 / Flask / Requests

1. Justificativa da Tecnologia Escolhida

Para a implementação deste servidor Web Proxy didático com controle de conteúdo, a opção adotada foi a combinação da linguagem Python com o microframework Flask e a biblioteca de requisições HTTP Requests.

A escolha do ecossistema Python + Flask justifica-se por três fatores fundamentais:

- **Agilidade e Legibilidade de Código:** Python oferece uma sintaxe limpa e expressiva, permitindo estruturar regras de filtragem de texto, manipuladores de arquivos JSON e rotas do proxy sem poluição boilerplate de código.
- **Gerenciamento Eficiente de Rotas Dinâmicas no Flask:** O Flask viabilizou a criação de uma interface de usuário interativa em HTML/CSS na raiz (`/`), ao mesmo tempo em que disponibiliza suporte a rotas dinâmicas como `/proxy` e `/<path:target_url>`, capturando o parâmetro de entrada da URL alvo com extrema facilidade.
- **Poder de Abstração da Biblioteca Requests:** Em comparação à implementação manual com sockets TCP puros, a biblioteca `requests` lida nativamente com o parsing de cabeçalhos HTTP complexos, decodificação de caracteres UTF-8 e gerenciamento de cookies/redirecionamentos, permitindo focar o desenvolvimento nas regras de negócio (bloqueio, filtragem e logging).

Em contrapartida, a utilização de Sockets TCP de baixo nível exigiria o parsing manual exaustivo dos cabeçalhos HTTP brutos e da codificação dos pacotes, o que aumentaria a complexidade de manutenção sem adicionar ganho didático ao controle de conteúdo solicitado.

2. Decisões de Implementação e Arquitetura

A aplicação foi estruturada seguindo o padrão de projeto modularizado, garantindo alta separação de responsabilidades. O código-fonte foi dividido em uma camada de servidor/rotas (app.py) e uma camada de utilitários (utils/proxy.py).

2.1 Fluxo de Funcionamento e Regras de Negócio

Sempre que uma URL é solicitada ao proxy, o sistema executa sequencialmente o seguinte fluxo:

1. **Normalização de Domínio e Checagem de Bloqueio (`blocked.json`):** O proxy extrai o host da URL e remove o prefixo `www.` através da função `normalize_domain()`. Caso o domínio esteja contido na lista negra, a requisição externa é abortada, o log é gravado como 'bloqueado' e a página customizada `templates/blocked.html` é retornada com o código HTTP `403 Forbidden`.
2. **Requisição ao Servidor de Origem:** Caso o site seja permitido, o proxy efetua a requisição ao servidor de origem via HTTP/HTTPS. Para evitar que respostas compactadas em Gzip ou Brotli impeçam a substituição de strings em texto puro, o cabeçalho `Accept-Encoding` é removido da requisição.
3. **Filtro de Conteúdo Case-Insensitive (`words.json`):** Se o `Content-Type` retornado for do tipo `text/html`, a função `filter_words()` inspeciona o texto HTML e aplica Expressões Regulares (`re.compile` com `re.IGNORECASE`) para substituir as palavras cadastradas pelas alternativas configuradas, registrando o log como 'filtrado'.
4. **Registro de Log Automático (`log.json`):** Todas as requisições válidas são registradas em `log.json` contendo obrigatoriamente: timestamp (data/hora), URL solicitada e a ação executada (permitido, bloqueado ou filtrado).

2.2 Desafios Técnicos e Surpresas Encontradas

Durante o desenvolvimento, destacaram-se dois grandes desafios técnicos:

- **Problema de Caracteres Corrompidos por Compactação:** Inicialmente, requisições para determinados domínios retornavam bytes binários compactados em Gzip. A solução foi interceptar e remover o cabeçalho `Accept-Encoding` da requisição original enviada pelo cliente, forçando o servidor de origem a entregar o HTML em texto puro legível.
- **Requisições Automáticas de Sub-recursos e Favicon:** Os navegadores solicitam automaticamente rotas como `/favicon.ico` e sub-recursos assíncronos que poluíam o `log.json` com erros de resolução de DNS. Foi implementado o descarte de domínios mal formatados e uma rota dedicada `/favicon.ico` que retorna `204 No Content`.

3. Explicação Técnica sobre o Protocolo HTTPS

Uma das principais reflexões deste trabalho consiste em compreender a atuação do Web Proxy diante do protocolo HTTPS e por que o filtro de substituição de conteúdo não consegue atuar de forma transparente em conexões cifradas.

3.1 Por que o Filtro de Conteúdo Não Funciona em Sites HTTPS?

O protocolo HTTPS utiliza a camada de criptografia TLS/SSL (*Transport Layer Security*) para estabelecer um canal seguro de ponta a ponta entre o cliente (navegador) e o servidor de origem. Esse mecanismo impede a atuação do filtro por dois motivos principais:

- **Criptografia do Payload:** Em uma conexão HTTPS nativa e cifrada, a carga útil da mensagem (o código HTML, textos e recursos) trafega de forma indecifrável para intermediários da rede. O proxy enxerga apenas um fluxo de bytes binários cifrados. Qualquer tentativa do proxy de alterar uma string no meio desse fluxo corrompe a integridade da mensagem e encerra a conexão por quebra de assinatura digital.
- **Falha na Carga de Sub-recursos Seguros (Layout Quebrado):** No nosso proxy em Python, embora a biblioteca `Requests` consiga baixar o HTML inicial da Wikipédia atuando como cliente TLS, ao devolver o HTML para o navegador local sem a devida interceptação de certificados, o navegador bloqueia a busca dos arquivos CSS e JavaScript secundários (carregados sob URLs HTTPS externas). Esse bloqueio por políticas de segurança (*Mixed Content* e HSTS) faz com que a página seja renderizada em texto puro, sem nenhuma formatação de estilo.

3.2 O que seria necessário para o Filtro Funcionar em HTTPS?

Para que um Web Proxy corporativo consiga inspecionar e filtrar o conteúdo de páginas HTTPS sem quebrar o layout da aplicação nem violar os alertas de segurança do navegador, seria necessário implementar uma técnica conhecida como **Inspeção TLS / SSL (Man-In-The-Middle Benigno)**:

1. **Certificado Raiz Próprio:** O proxy deve gerar uma Autoridade Certificadora (CA) própria e o seu Certificado Raiz deve ser previamente instalado e marcado como confiável no navegador/sistema operacional do usuário.
2. **Interceptação e Re-criptografia:** Ao receber uma requisição HTTPS, o proxy abre a conexão TLS com o servidor de origem, decifra o HTML na memória, aplica o filtro de substituição do `words.json`, assina a nova resposta utilizando o seu próprio certificado digital e a envia ela cifrada de volta ao navegador do cliente.

4. Aprendizados sobre o Protocolo HTTP

A construção prática deste Web Proxy proporcionou consolidação teórica e prática sobre os fundamentos da arquitetura da Web:

- **Duplo Papel do Proxy:** Foi possível vivenciar na prática como um servidor proxy atua concomitantemente como Servidor HTTP (recebendo conexões na porta 5000) e como Cliente HTTP (disparando requisições externas para os servidores de destino).
- **Importância dos Cabeçalhos (Headers):** Compreendeu-se o impacto de cabeçalhos como `Host`, `Content-Type`, `Accept-Encoding` e `Content-Length` no controle de fluxo, percebendo como a remoção ou adição incorreta de um cabeçalho pode corromper a exibição do documento.
- **Semântica dos Códigos de Status:** A aplicação praticou a utilização correta dos códigos HTTP:
  - `200 OK`: Repasse/filtro efetuado com sucesso.
  - `403 Forbidden`: Bloqueio de domínios restritos.
  - `204 No Content`: Recursos secundários ignorados.
  - `502 Bad Gateway`: Falhas de conexão remota.

5. Relato do Uso de Ferramentas de Inteligência Artificial

Em conformidade com as diretrizes de transparência do edital da disciplina, declara-se que ferramentas de Inteligência Artificial Generativa foram utilizadas durante o desenvolvimento do trabalho como apoio ao aprendizado e refatoração de código.

- **Ferramenta Utilizada:** Google Gemini.
- **Forma de Uso:** A IA foi consultada para depuração de erros de ambiente virtual (`venv`) e Git/HTTPS, estruturação da rotina de substituição case-insensitive de Expressões Regulares (`re.IGNORECASE`), unificação de estilos CSS entre as páginas e fundamentação teórica sobre os limites do protocolo HTTPS e HSTS.
- **Modificações e Aprendizado:** Todo o código gerado foi revisado, testado e adaptado manualmente. A estrutura de arquivos foi refatorada manualmente criando o pacote `utils/proxy.py`, a lógica de normalização de domínios foi ajustada para lidar com prefixos `www.` e a integração com os templates HTML foi personalizada para atender integralmente aos requisitos do professor.